

1.4 Vector Scalar Multiplication



Core skills you will master in this section:

- ▶ Scalar multiplication changes a vector's length and direction: a positive scalar scales it in the same direction, a negative scalar scales it in the opposite direction, and zero turns it into the zero vector.
- ▶ A geometric understanding of how scalar multiplication stretches or shrinks a vector.
- ▶ Linear combinations: generating a new vector as a weighted sum of several vectors of the same dimension.
- ▶ Two linearly independent vectors: noncollinear vectors span an entire plane (through the origin).
- ▶ Two linearly dependent vectors: collinear vectors span only a one-dimensional line (through the origin).

This section introduces scalar multiplication of vectors and its geometric meaning, and uses code and diagrams to show how vector transformations are implemented and visualized on a computer. Geometrically, scalar multiplication is a stretching or shrinking transformation of a vector. In RGB color space, scalar multiplication of a vector can change the shade of a color: the smaller the scalar, the darker the color, approaching black. We also introduce linear combinations of vectors. A linear combination is the process of generating a new vector as a weighted sum of several vectors, using scalars as weights. Building on this, the section also compares the two concepts of linear independence and linear dependence.

Scalar Multiplication

Scalar multiplication of a vector means multiplying each component of the vector by the same scalar to obtain a new vector.

For example, given a nonzero n -dimensional column vector $\mathbf{a}$ and a scalar k , their product is defined as

$$k\mathbf{a} = k \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} ka_1 \\ ka_2 \\ \vdots \\ ka_n \end{bmatrix}$$

Code 1 computes scalar multiplication of a vector. Let's walk through the key lines below.

a This defines a one-dimensional array representing a vector. In NumPy, the `numpy.array()` function is used to create arrays, which can be one-dimensional, two-dimensional, or higher-dimensional matrices.

b This uses `numpy.linalg.norm()` to compute the length of the vector `a_vec` (its magnitude, modulus, L2 norm, or Euclidean norm). In NumPy, `np.linalg.norm()` is a function

for computing the magnitude of a vector or matrix. By default, if the input is a vector, the function computes the Euclidean norm, i.e., the length of the vector.

c This computes the product of `a_vec` and `k`. NumPy allows a scalar to be multiplied directly with an array, meaning every component of the vector is multiplied by `k`.

d This calls `numpy.linalg.norm()` again to compute the length of `k_a_vec`.

Code 1. Scalar Multiplication of a Vector |  LA_01_04_01.ipynb

```
## 初始化
import numpy as np

## 定义向量
a a_vec = np.array([3, 4])
b np.linalg.norm(a_vec)

## 标量乘法
k = -2
c k_a_vec = k * a_vec
d np.linalg.norm(k_a_vec)
```

Geometric Perspective

Geometrically, scalar multiplication of a vector is a stretching or shrinking transformation: a positive scalar stretches or shrinks the vector in the same direction, while a negative scalar does so in the opposite direction. In fact, scalar multiplication never introduces a new direction — it only rescales the vector's length by a factor of $|k|$, and the sign of k decides whether the original direction is kept or reversed.

Clearly, the new vector $k\mathbf{a}$ is collinear with the original nonzero vector $\mathbf{a}$, as mentioned in the previous section, collinear means two vectors lie on the same line, pointing in the same or opposite directions.

When $|k| > 1$, the vector's length increases. As shown in Figure 1 (a), when $k > 1$, the new vector $k\mathbf{a}$ points in the same direction as the original vector $\mathbf{a}$ and its length increases; as shown in Figure 1 (b), when $k < -1$, $k\mathbf{a}$ and $\mathbf{a}$ point in opposite directions, and the length increases.

When $0 < |k| < 1$, the vector shortens; as shown in Figure 1 (c), when $0 < k < 1$, the new vector $k\mathbf{a}$ points in the same direction as the original vector $\mathbf{a}$ and its length shortens; as shown in Figure 1 (d), $k\mathbf{a}$ and $\mathbf{a}$ point in opposite directions, and the length shortens.

Now let's look at two special values.

As shown in Figure 1 (e), when $k = -1$, the new vector $k\mathbf{a}$ is the opposite vector of the original vector $\mathbf{a}$.

As mentioned in the previous section, the opposite vector is the new vector obtained by negating every component of a vector.

Geometrically, the opposite vector points in the direction opposite to the original vector but has the same length; it is symmetric to the original vector about the origin.

As shown in Figure 1 (f), when $k = 0$, the new vector $k\mathbf{a}$ is the zero vector $\mathbf{0}$.

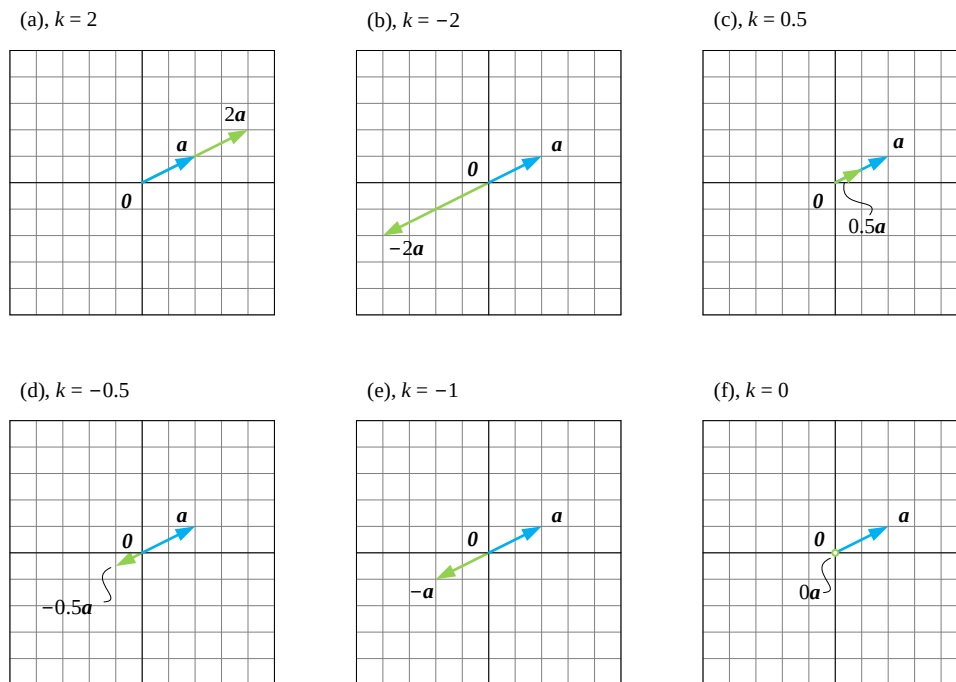


Figure 1. Scaling a Vector

Code 2 uses the `matplotlib.pyplot.arrow()` function to visualize scalar multiplication of a vector. Let's walk through the key lines below.

a and **b** both use Matplotlib's `matplotlib.pyplot.arrow()` function to draw an arrowed vector in the 2D coordinate system. The `matplotlib.pyplot.arrow()` function requires us to provide the starting coordinates, ending coordinates, and arrow style parameters.

In **a**, `0, 0` represents the starting point of the arrow, while `a_vec[0]`, `a_vec[1]` represent the x- and y-direction components of the arrow, i.e., the coordinates of vector a .

The parameter `head_width=0.2` sets the width of the arrowhead, making it clearer in the figure, and `head_length=0.3` sets the length of the arrowhead, making it easier to identify.

The parameter `fc='#00B0F0'` specifies the arrow's face color, blue in this case, and `ec='#00B0F0'` specifies the arrow's edge color, matching the face color to keep the arrow's color consistent.

The parameter `label="a"` attempts to add a legend entry for the arrow.

In Matplotlib, colors can be specified in many ways, including hexadecimal color codes, RGB colors, HTML color names, single-character abbreviations, grayscale levels, colormaps, and more. In hexadecimal color codes, `'#FF0000'` represents pure red, `'#00FF00'` pure green, `'#0000FF'` pure blue, and `'#808080'` a shade of gray.

Please annotate the remaining lines of Code 2 line by line.

Code 2. Visualizing Scalar Multiplication of a Vector |  LA_01_04_02.ipynb

```

## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 定义向量
a_vec = np.array([2, 1])

## 标量乘法
k = -2
k_a_vec = k * a_vec

## 可视化
plt.figure(figsize=(6,6))

# 绘制向量 a
plt.arrow(0, 0, a_vec[0], a_vec[1],
          head_width=0.2, head_length=0.3,
          fc='#00B0F0', ec='#00B0F0', label="a")

# 绘制向量 k * a
plt.arrow(0, 0, k_a_vec[0], k_a_vec[1],
          head_width=0.2, head_length=0.3,
          fc='#92D050', ec='#92D050', label="k*a")

# 装饰
plt.gca().set_aspect('equal', adjustable='box')
plt.xlim(-5, 5) # 设置 x 轴范围
plt.ylim(-5, 5) # 设置 y 轴范围
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.xticks(np.arange(-5,6))
plt.yticks(np.arange(-5,6))
plt.grid(color='gray', alpha=0.8, linestyle='--', linewidth=0.25)
plt.legend()

```

Darkening RGB Colors

Back in RGB color space, as shown in Figure 2, the red vector $\mathbf{e}_1$, green vector $\mathbf{e}_2$, and blue vector $\mathbf{e}_3$ are each multiplied by a scalar k ($0 < k < 1$); as k approaches 0, the vectors (the scatter points) keep shortening, and the corresponding colors keep darkening. However, the vector directions stay unchanged, meaning the hue stays the same.

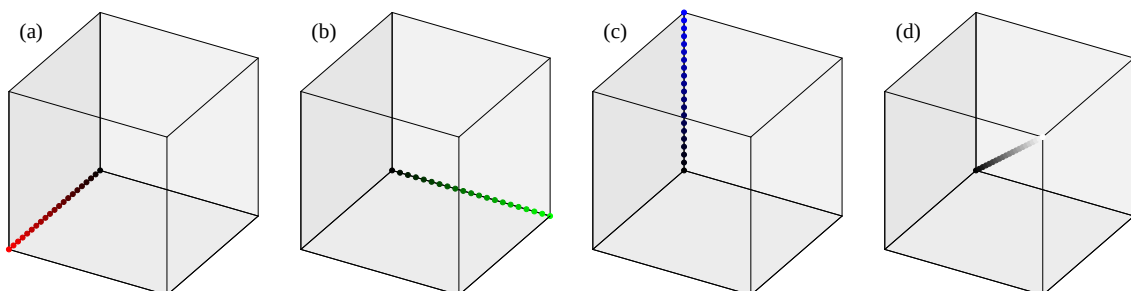


Figure 2. Darkening Red, Green, Blue, and White

We know that in RGB color space, the 3-dimensional all-ones column vector, $[1, 1, 1]^T$ corresponds to white. When we multiply the all-ones vector by a scalar k ($0 < k < 1$),

$$k(\mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3) = k \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} k \\ k \\ k \end{bmatrix}$$

the resulting new color represents a shade of gray; as k approaches 1, the gray becomes lighter and the color approaches white, while as k approaches 0, the color gradually approaches black. At $k = 0$ we get black, and at $k = 1$ we get white.

Thus, by gradually adjusting the value of the scalar k , we can achieve a smooth gradient from black to white across the grayscale range. Written as $(1 - k) \cdot \text{black} + k \cdot \text{white}$, this blend is an early instance of a convex combination — a weighted sum whose weights are nonnegative and sum to 1 — an idea later chapters revisit.

Figure 3's two subplots each define a set of colors in a different way and plot the corresponding sequence of scatter points based on those colors, with each group of points darkening progressively. The color change is achieved through scalar multiplication: each base color is multiplied by a continuously varying k ($0 < k < 1$), causing the color to transition gradually from its original shade to a darker one.

By assigning different color vectors in RGB space, either following a fixed rule or at random, and then scaling them with a gradually varying scalar between 0 and 1, we can create rich gradient effects, letting a single hue display many different levels of brightness.

 Think about how to make a color lighter instead.

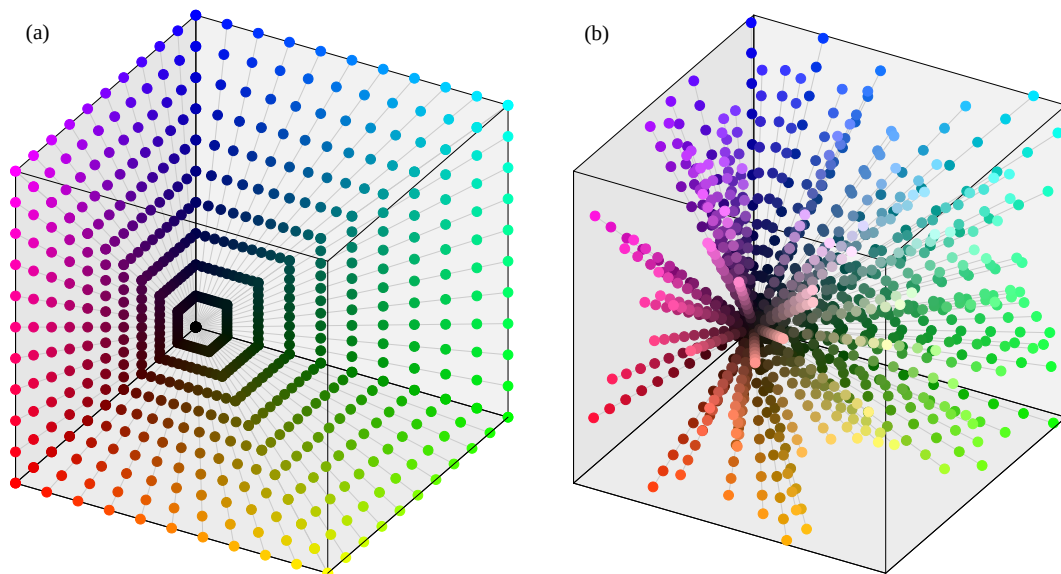


Figure 3. Darkening Colors

Linear Combinations

In vector operations, combining scalar multiplication with vector addition can generate a much broader space.

Given two nonzero vectors of the same dimension, $\mathbf{a}$ and $\mathbf{b}$, we can form their weighted sum using scalars k and t , namely

$$\mathbf{c} = k\mathbf{a} + t\mathbf{b}$$

where k and t are real numbers.

The equation above is, in fact, a linear combination of $\mathbf{a}$ and $\mathbf{b}$. A linear combination is a new vector constructed as the weighted sum of a set of vectors and their corresponding scalars (coefficients).

Geometrically, by adjusting the values of k and t , we can generate a line through the origin (when $\mathbf{a}$ and $\mathbf{b}$ are collinear), or fill an entire plane (when $\mathbf{a}$ and $\mathbf{b}$ are not collinear).

Let's analyze these two cases below.

Linear Independence

If two nonzero vectors $\mathbf{a}$ and $\mathbf{b}$ are not collinear — that is, there is no scalar k such that $\mathbf{a} = k\mathbf{b}$ (or, equivalently, $\mathbf{b} = k\mathbf{a}$) holds — we say the nonzero vectors $\mathbf{a}$ and $\mathbf{b}$ are **linearly independent**.

In other words, $\mathbf{a}$ and $\mathbf{b}$ cannot be expressed as scalar multiples of each other, so geometrically they do not lie on the same line.

Geometrically, $\mathbf{a}$ and $\mathbf{b}$ span a two-dimensional plane, which can be written as $\text{span}(\mathbf{a}, \mathbf{b})$. This book introduced the concept of span earlier.

Simply put, the span of a set of vectors is the set of all their linear combinations — that is, the set of all vectors that can be generated from them through addition and scalar multiplication. A span is always a vector subspace in its own right: it contains the zero vector and is closed under both addition and scalar multiplication, a property later chapters will use extensively.

Let's look at two sets of examples.

As shown in Figure 4, in all four subplots $\mathbf{a}$ and $\mathbf{b}$ are noncollinear, i.e., linearly independent. If k and t range over all real numbers, $\mathbf{a}$ and $\mathbf{b}$ can span the entire plane. That is, for linearly independent, nonzero, two-dimensional vectors $\mathbf{a}$ and $\mathbf{b}$, **every vector in the plane can be expressed as a linear combination $k\mathbf{a} + t\mathbf{b}$** .

In particular, if k and t are restricted to integers, the endpoints of the vector $k\mathbf{a} + t\mathbf{b}$ land on the intersections of a regular grid; these grid lines share a common feature: they are parallel to one another, evenly spaced, and pass through the origin.

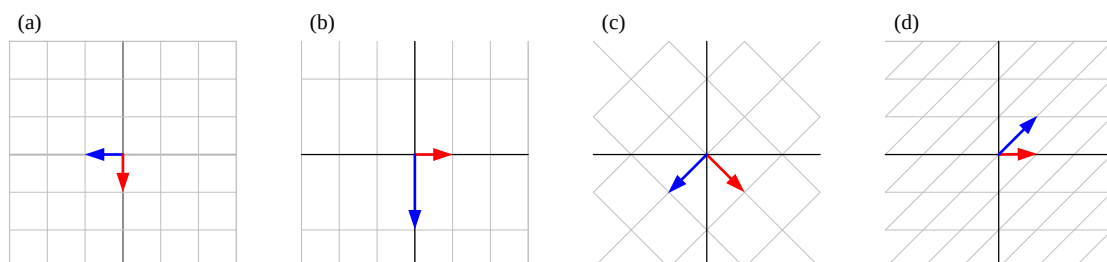
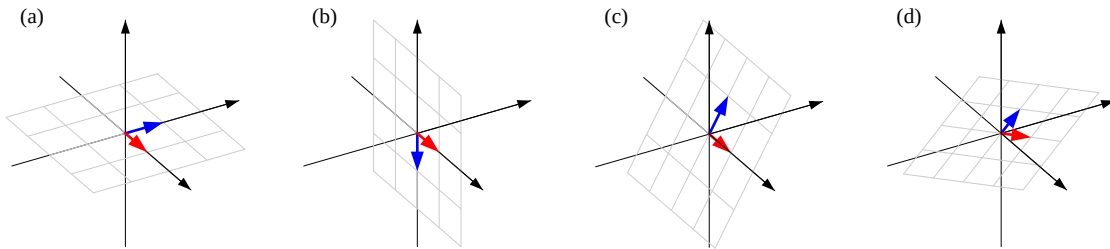


Figure 4. $\mathbf{a}$ and $\mathbf{b}$ Weaving a Planar Grid, $\mathbf{a}$ and $\mathbf{b}$ as 2D Vectors

As shown in Figure 5, two noncollinear (linearly independent) 3-dimensional vectors $\mathbf{a}$ and $\mathbf{b}$ still span a two-dimensional plane (through the origin) in three-dimensional space. Linear independence means $\mathbf{a}$ and $\mathbf{b}$ cannot be expressed as a single scalar multiple of each other — each contributes a genuinely "independent" new direction.

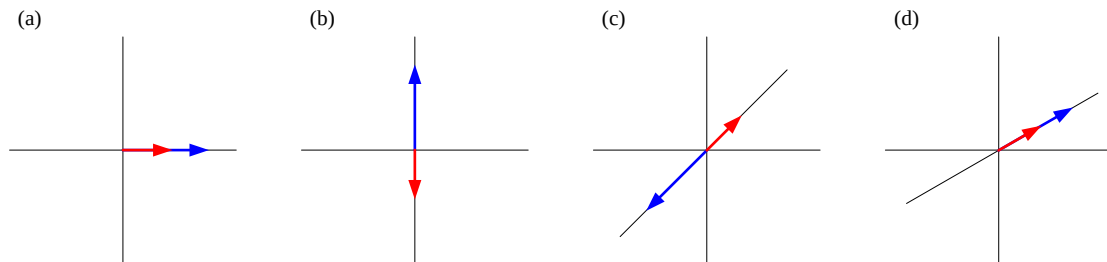
Figure 5. $\mathbf{a}$ and $\mathbf{b}$ Weaving a Planar Grid, $\mathbf{a}$ and $\mathbf{b}$ as 3D Vectors

Linear Dependence

However, when $\mathbf{a}$ and $\mathbf{b}$ are collinear — also called linearly dependent — the linear combinations of $\mathbf{a}$ and $\mathbf{b}$ can only vary along the single line through the origin that $\mathbf{a}$ and $\mathbf{b}$ lie on, and cannot extend into a higher-dimensional space.

In other words, every vector obtained as a linear combination of $\mathbf{a}$ and $\mathbf{b}$ still lies on the line they share, and cannot cover the entire two-dimensional space.

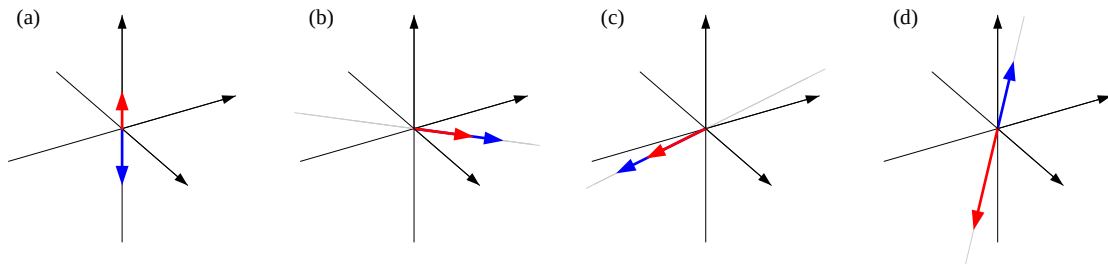
As shown in Figure 6, the two-dimensional nonzero vectors $\mathbf{a}$ and $\mathbf{b}$ are collinear; they span only a line (through the origin), not the entire plane.

Figure 6. $\mathbf{a}$ and $\mathbf{b}$ Collinear, $\mathbf{a}$ and $\mathbf{b}$ as 2D Nonzero Vectors

As shown in Figure 7, in three-dimensional space, if two nonzero vectors $\mathbf{a}$ and $\mathbf{b}$ are linearly dependent, their linear combinations still vary only along a single line through the origin. No matter how the scalar coefficients k and t are adjusted, every vector generated by $k\mathbf{a} + t\mathbf{b}$ is confined to this line and cannot span a plane.

Intuitively, $\mathbf{b}$ is simply a multiple of $\mathbf{a}$, so they point in the same or opposite directions and, in essence, provide no additional dimensional information. Such a pair of vectors lacks independence and cannot be used to construct a higher-dimensional spatial structure.

This book will introduce these vector space concepts in detail later; here they are only a preview.

Figure 7. $\mathbf{a}$ and $\mathbf{b}$ Collinear, $\mathbf{a}$ and $\mathbf{b}$ as 3D Nonzero Vectors

Please note that the dimension of a vector and the dimension of the space it spans are two different concepts.

Consider a two-dimensional nonzero vector, say $\mathbf{x} = [x_1, x_2]^T$; it must be displayed in the two-dimensional plane, but the space it spans, $\text{span}(\mathbf{x})$, is "one-dimensional," because all its scalar multiples lie on the same line (through the origin). No matter how much this vector is stretched or shrunk, it still lies along its original direction (or the opposite one) and never extends to the whole plane.

Similarly, consider a three-dimensional nonzero vector, say $\mathbf{x} = [x_1, x_2, x_3]^T$; although it must be visualized in three-dimensional space, it is still one-dimensional. All its scalar multiples lie along a fixed direction (or its opposite), forming a line through the origin rather than filling all of three-dimensional space.

It's like a thin straight line stretched across a room: no matter how far either end extends, it remains merely "one-dimensional" and never becomes a surface or a solid.

The dimension of a vector tells us how many components it has, while the dimension of the space it occupies determines how we understand and display that vector geometrically.

Linear Combinations of Colors

In RGB color space, a linear combination is simply a mixture of colors; let's work through some examples below.

The linear combination of pure red $\mathbf{e}_1$ and pure green $\mathbf{e}_2$ is written as

$$k_1 \mathbf{e}_1 + k_2 \mathbf{e}_2 = k_1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + k_2 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

As shown in Figure 8 (a), k_1 and k_2 are generated by a random number generator. The random numbers k_1 and k_2 follow a continuous uniform distribution on $[0, 1]$. Simply put, a continuous uniform distribution means that within an interval, every point has exactly the same probability of being chosen, with no bias.

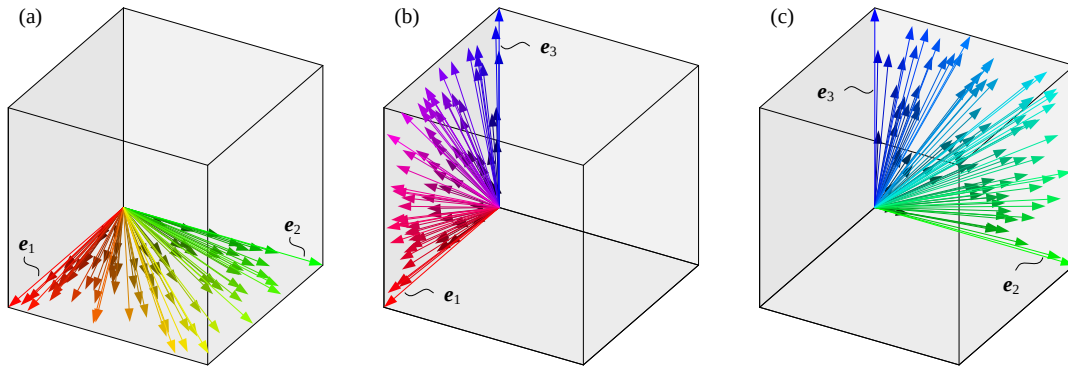


Figure 8. Linear Combination of Two Colors

We can imagine that as k_1 and k_2 range over all possible real numbers, the vector formed by $k_1\mathbf{e}_1 + k_2\mathbf{e}_2$ spans the entire x_1x_2 plane.

As shown in Figure 8 (b), the linear combination of pure red $\mathbf{e}_1$ and pure blue $\mathbf{e}_3$ is

$$k_1\mathbf{e}_1 + k_2\mathbf{e}_3 = k_1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + k_3 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

The random numbers k_1 and k_3 likewise follow a continuous uniform distribution on $[0, 1]$. Please analyze Figure 8 (c) on your own.

The weighted mixture of pure red $\mathbf{e}_1$, pure green $\mathbf{e}_2$, and pure blue $\mathbf{e}_3$ produces the colorful RGB color space shown in Figure 9.

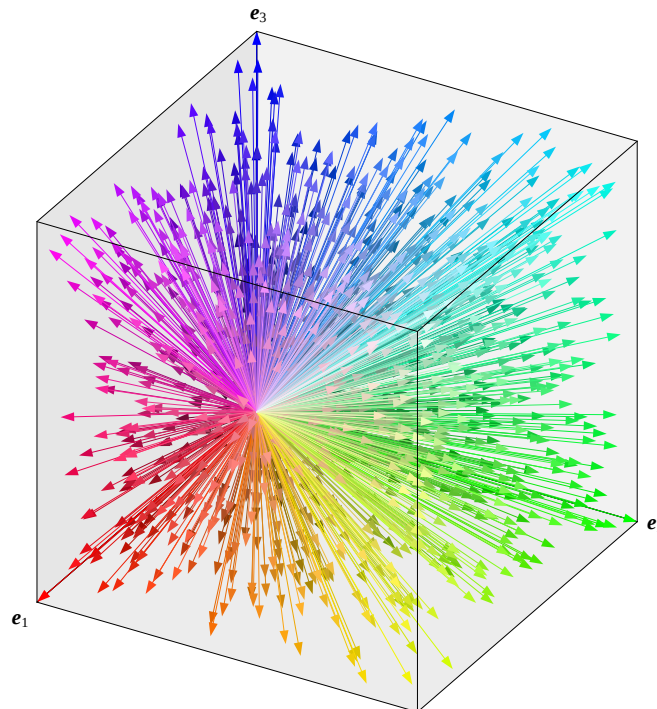


Figure 9. Linear Combination of Three Color Vectors



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ are all nonzero 2-dimensional vectors. If the three are linearly dependent, what does that look like geometrically?

Q2. $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ are all nonzero 3-dimensional vectors. If the three are linearly dependent, what does that look like geometrically?

Q3. Mix the pure red vector $[1, 0, 0]^T$ with the dark red vector $[1/2, 0, 0]^T$. What are the characteristics of the resulting color?

Q4. Mix the white vector $[1, 1, 1]^T$ with the gray vector $[1/2, 1/2, 1/2]^T$. What are the characteristics of the resulting color?

Q5. Try writing Python code to plot Figure 3 (a), this scatter plot.

Q6. Try writing Python code to plot Figure 9, this arrow plot.

Q7. Which of the following sets of vectors can have linear combinations that "span" the entire plane?

▶ $\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

▶ $\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \end{bmatrix}$